In Android apps, the folders are split under the Java folder into application code; the presentation layer tests, which sit under the androidTest folder; and the logic layer tests, which live under the test folder. The Pact tests go with the logic tests in the test folder. Once the test file has been created, the class needs annotating with the Pact extension annotation, as follows.

Listing 5.8 Movie Pact class

```
import au.com.dius.pact.consumer.junit5.PactConsumerTestExt
import org.junit.jupiter.api.TestInstance
import org.junit.jupiter.api.extension.ExtendWith

@ExtendWith(PactConsumerTestExt::class)
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
class MoviesApiPactTest {
```

The `ExtendWith` annotation takes the `PactConsumerTestExt` class as an argument to allow the test to enable Pact setup tasks. The Pact setup includes starting the mock server and methods to check that the class has all the right elements for the test to run successfully. `PactConsumerTestExt` comes bundled with the Pact JUnit 5 package, working with `ExtendWith` from the Junit Jupiter API package. Testing instance lifecycle `PER_CLASS` shares the test instance state between the test methods.

Once the class has been annotated, the next step is to build our Pact contract for the mock server when intercepting the request. To build the Pact contract, start with a `createPact` function. Remember from chapter 3, the consumer diagram shown in figure 5.3 is what we're going to set up in this chapter.

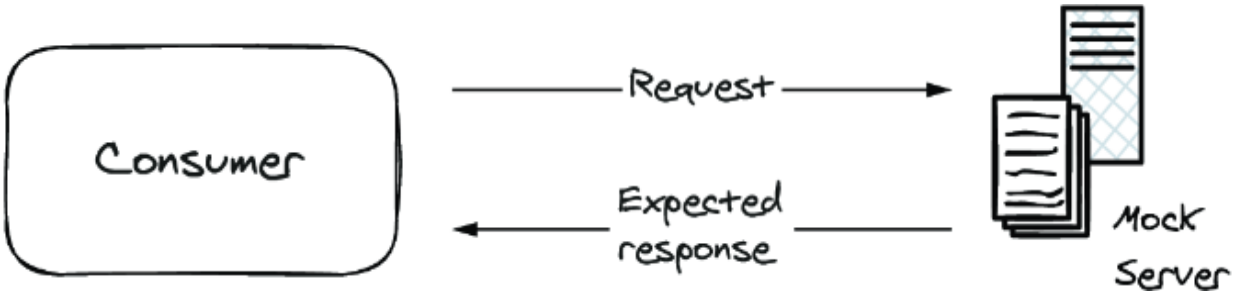


Figure 5.3 The consumer sets up the request and response in the form of a contract, which gets created within a mock server.